

Financial Frictions, Used Capital, and Misallocation

Replication package - Shadi Rahimfallah

ATTRIBUTION

The solution method follows the computational approach of Midrigan, V. and D. Y. Xu (2014), "Finance and Misallocation: Evidence from Plant-Level Data," American Economic Review 104(2), 422-458, with function approximation from Miranda and Fackler's CompEcon toolbox. The model solved here, and the code that implements it, are written for this paper.

CONTENTS

tables.m	Tables 1 to 7	MATLAB, about 1 minute
figures.m	Figures 1 and 2	MATLAB, about 1 minute
empirical/ compute_data_moments.py	Tables 1 and 2 and the calibration targets	Python
benchmark/ model_with_no_used/ cf_calib_B/ cf_etap/	the model, with new and used capital the model without used capital, recalibrated without used capital, ability tail held at the benchmark without used capital, benchmark tail, nothing refitted	

REQUIREMENT: THE COMPECON LIBRARY

The MATLAB code requires Miranda and Fackler's CompEcon library on the MATLAB path. It is not redistributed here. Download it from

<http://www4.ncsu.edu/~pfackler/compecon/toolbox.html>

and add the CTools folder to your path before running anything:

```
addpath('<your path>/CompEcon/CTools')
```

HOW TO REPRODUCE THE RESULTS

1. Install CompEcon as above.
2. In MATLAB, from this folder:

```
>> tables      writes TABLES.txt      (Tables 1 to 7)
>> figures     writes figure1, figure2 and the standalone
               tilt panel, each as .pdf and .png
```

Both read the solved economies stored in the four model folders and re-solve nothing, so each finishes in about a minute.

3. For the empirical results, from empirical/:
\$ python compute_data_moments.py

Every number in the model tables is computed in tables.m from a stored .mat field or from an expression over stored fields. Nothing is hardcoded. TABLES.txt ends with a provenance appendix listing the source file, its size and its timestamp, and naming the field behind every row. FIGURES.txt prints the numbers behind every plotted curve, so any point on any figure can be checked against the picture.

WHICH FILE PRODUCES WHICH EXHIBIT

Tables 1-2	empirical/motivation/used_capital_table.py	
	echoed by tables.m	from the committed results file
Tables 3-7	tables.m	-> TABLES.txt
Figure 1	figures.m	-> figure1.pdf (two panels)
Figure 2	figures.m	-> figure2.pdf
	figures.m	-> figure_usedcapital_tilt.pdf
		Figure 1's right panel, on its own canvas

Table 7 reads two sensitivity sweeps stored in benchmark/:
robustness_phi_result.mat and robustness_gamma_result.mat, written by
robustness_phi.m and robustness_gamma.m. The discussion of the gamma
sweep is in benchmark/robustness_gamma_results.txt.

THE SURVEY DATA ARE NOT INCLUDED

The Viet Nam SME Survey is not included here. Download it from

<https://www.wider.unu.edu/database/viet-nam-sme-database>

The empirical code is included, and so is its output: every subfolder of empirical/ contains the .txt results it produced, so all numbers the paper takes from the survey can be checked without the data.

To run the empirical code yourself, place the six wave files in
empirical/enterprise_data/ as

enterprise 2005.dta ... enterprise 2015.dta

compute_data_moments.py detects them and runs everything; without them it
stops and lists the saved output instead. Cite the data as

CIEM, ILSSA, UCPH, and UNU-WIDER (year of survey). Viet Nam SME Survey.

One exception: empirical/r_target/ uses public World Bank series and runs
as shipped, with or without the survey.

CHECKING THAT THE STORED EQUILIBRIA ARE GENUINE

Not needed to reproduce any exhibit. From benchmark/:

```
>> verify_benchmark
```

It ignores every cache, reads the six calibrated parameters out of parameters_benchmark.m, re-solves the equilibrium from scratch on the production grid, and prints the recomputed moments beside the targets and beside the stored benchmark_result.mat. It takes about 40 minutes.

```
>> verify_benchmark(true)
```

 runs the same check on a coarse grid in a few minutes. That is a setup test, not a verification: do not report its numbers.

RE-SOLVING EVERYTHING FROM SCRATCH

Slow: each economy takes hours. Inside a model folder, in order,

```
>> nofriction           the frictionless twin
>> analysis_used_capital (benchmark)
>> analysis_cf          (the other three economies)
```

which overwrite the .mat files the two entry points read. analysis_* requires nofriction_result.mat, because each economy is compared with its own frictionless twin.

Recalibrating is longer again. >> recalibrate runs a Newton search over the six calibrated parameters, roughly seven solves per iteration, and is measured in days rather than hours.

SOFTWARE

MATLAB R2023b with the CompEcon library.

Python 3.11 with pandas, numpy and matplotlib.